

Recursion 2

"MOTIVATION MAY BE
WHAT STARTS YOU OFF,
BUT IT'S HABIT THAT KEEPS YOU
GOING BACK FOR MORE."

— MIYA YAMANOUCHI

STARTUPVITAMINS

Good
Morning
☺



Content

01. Quizzes
02. Tower of Hanoi Pure recursion
03. Generate parenthesis Backtracking

01. void solve (int N)

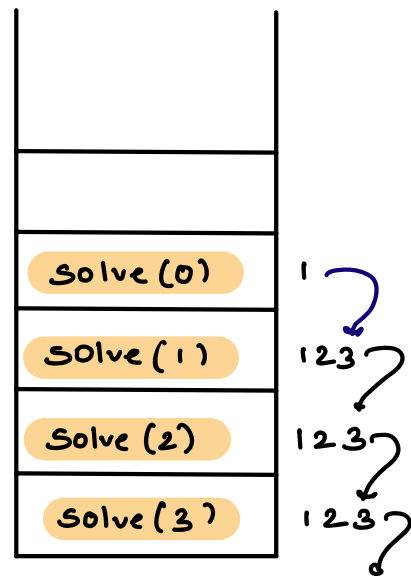
if (N==0) return ①

solve (N-1) ②

print (N) ③

3

Output = 1 2 3



02. void solve (int N)

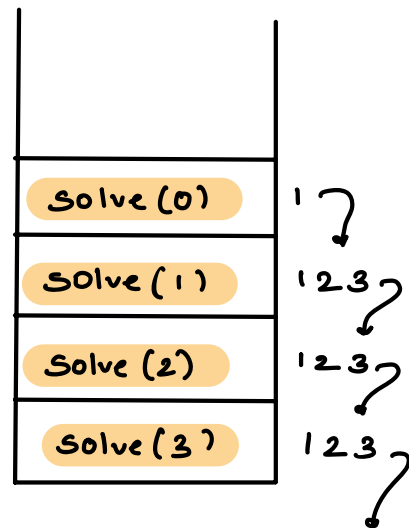
if (N==0) return 1

print (N) 2

solve (N-1) 3

3

Output = 3 2 1



03. void solve (int N) ↗ -3

if (N==0) return 1

print (N) 2

solve (N-1) 3

3

Output : Error → Stack overflow

solve (-7)

solve (-6)

solve (-5)

solve (-4)

solve (-3)

Tower Of Hanoi

There are n disks placed on tower A of different sizes

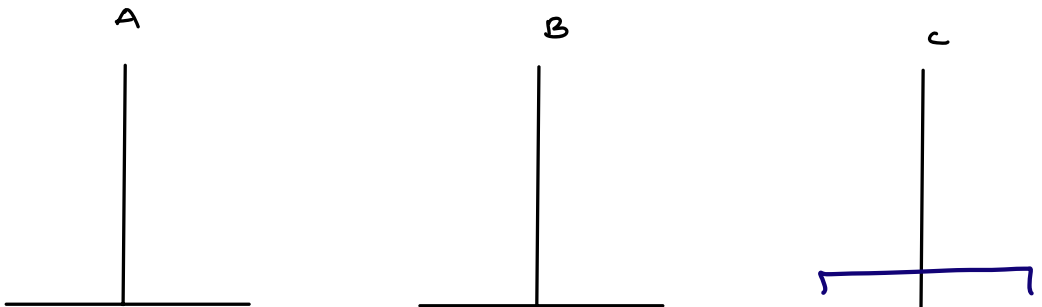
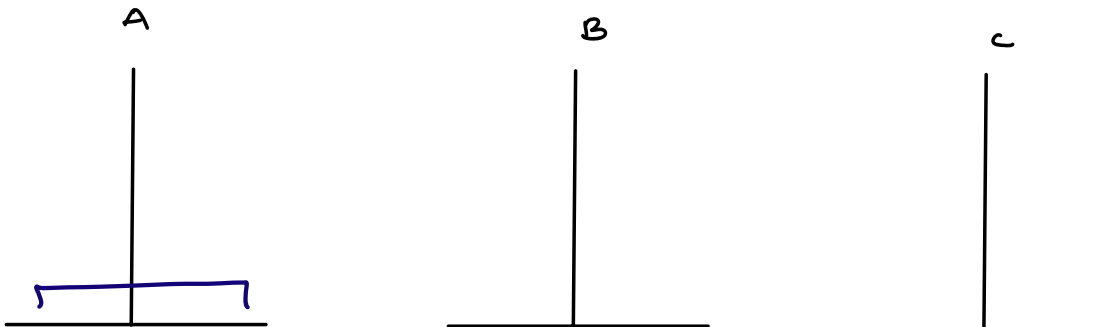
Goal → Move all disks from tower A to C using tower B if needed

Constraint →

01. Only 1 disk can be moved at a time
02. Larger disk can't be placed at smaller disk at any step.

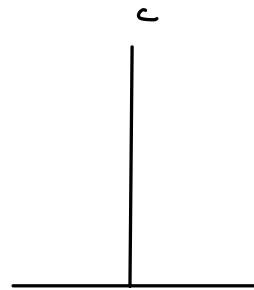
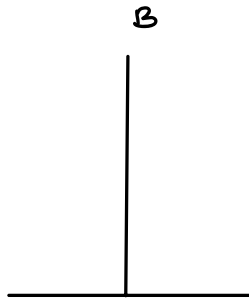
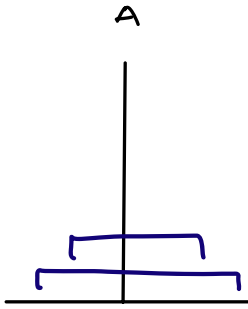
Print the movement of disks from A to C in minimum steps

$n=1$

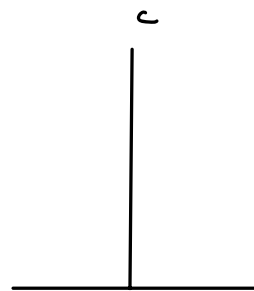
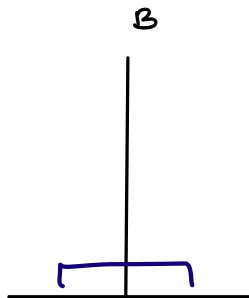
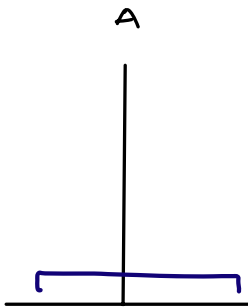


Output = 1 : A → C

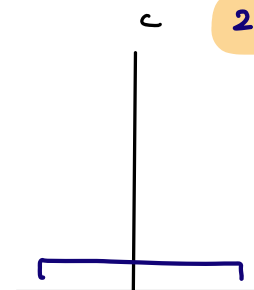
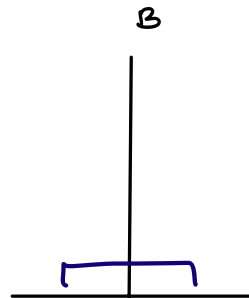
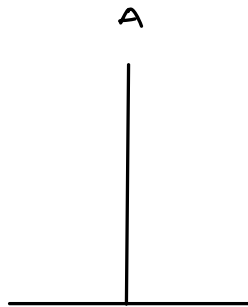
$n=2$



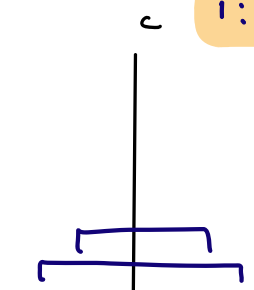
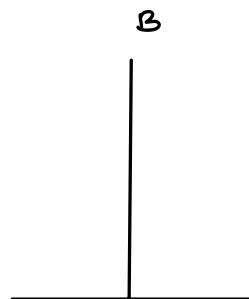
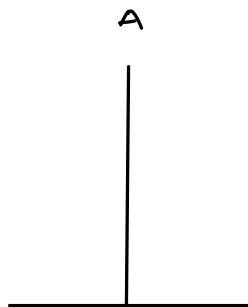
1: $A \rightarrow B$



2: $A \rightarrow C$



1: $B \rightarrow C$



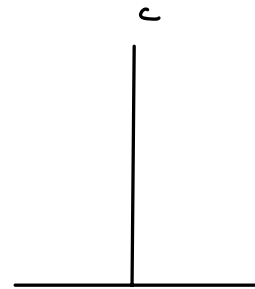
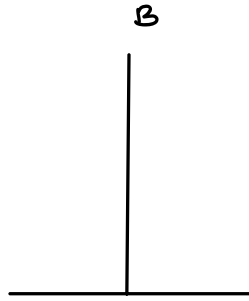
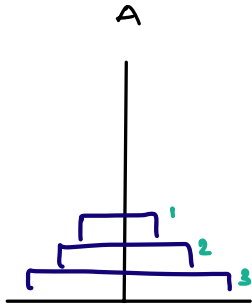
Output

1: $A \rightarrow B$

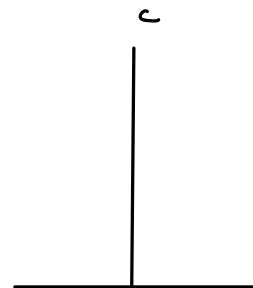
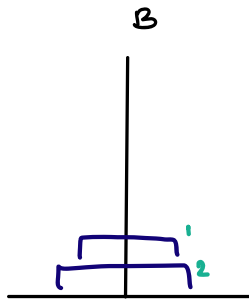
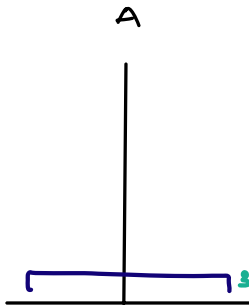
2: $A \rightarrow C$

1: $B \rightarrow C$

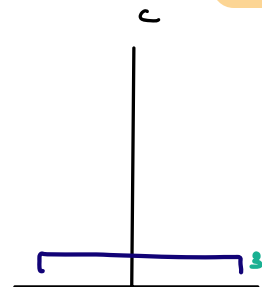
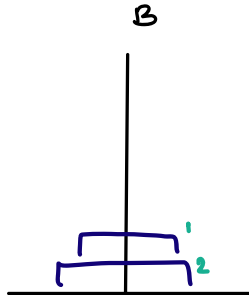
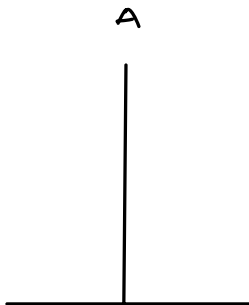
$n=3$



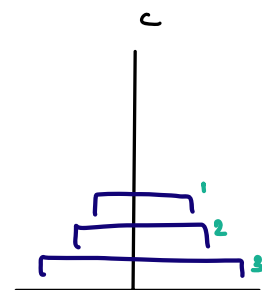
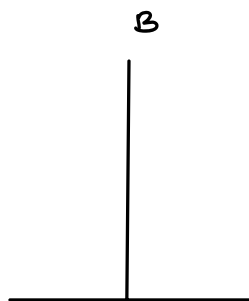
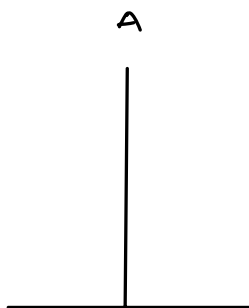
1: $A \rightarrow C$
2: $A \rightarrow B$
1: $C \rightarrow B$



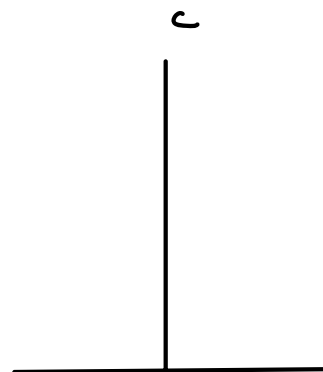
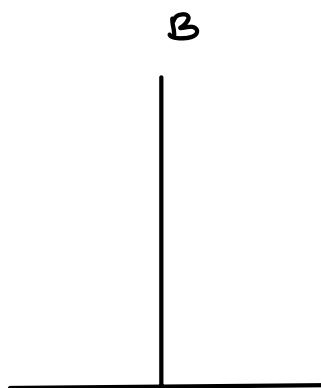
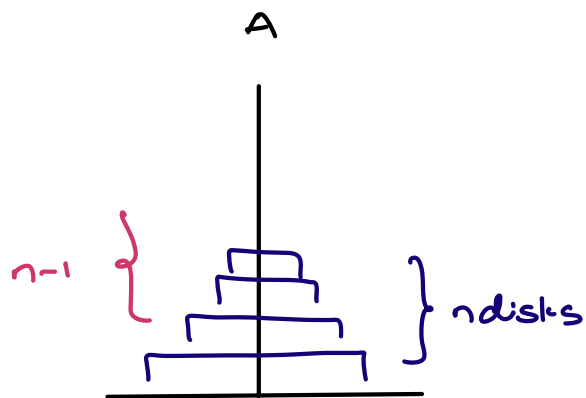
3: $A \rightarrow C$



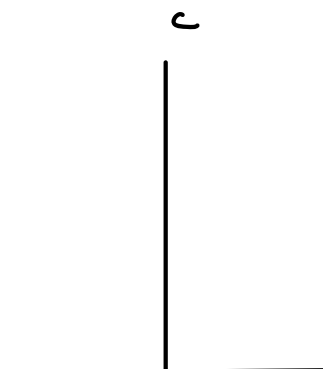
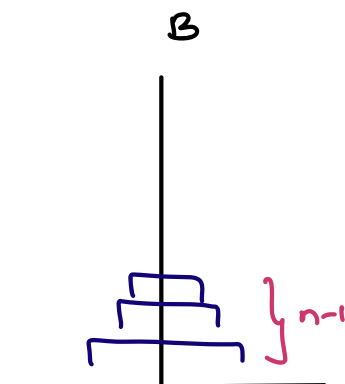
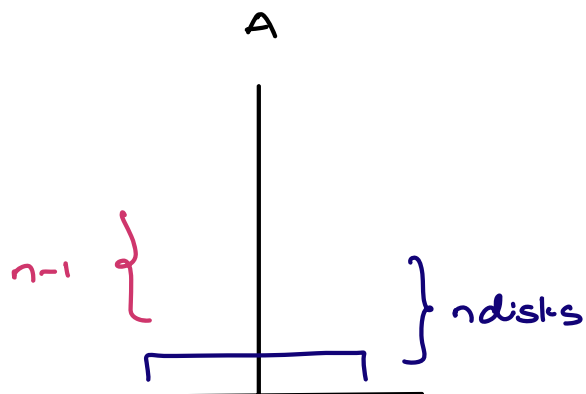
1: $B \rightarrow A$
2: $B \rightarrow C$
1: $A \rightarrow C$



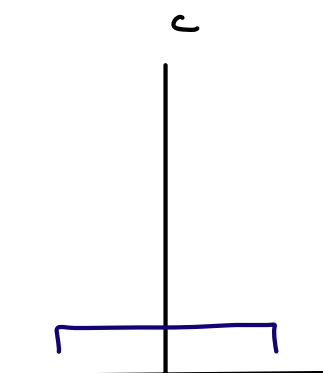
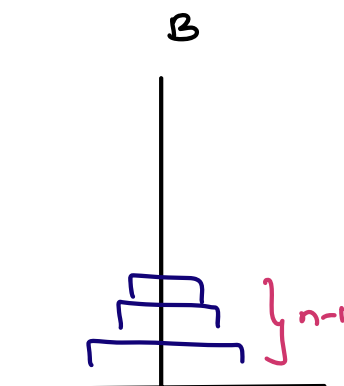
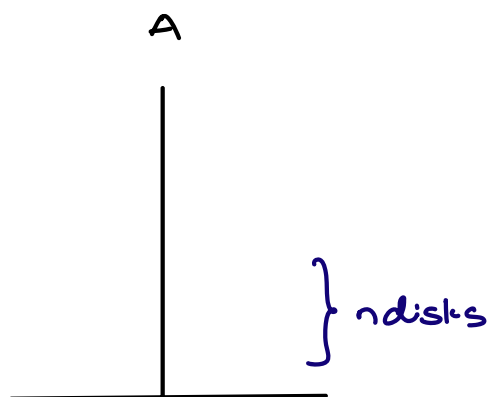
n disks



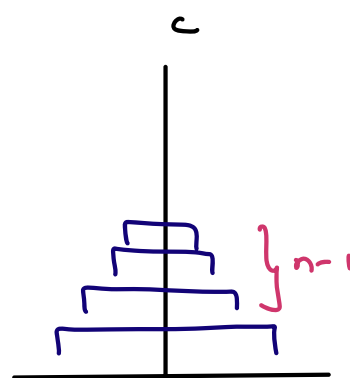
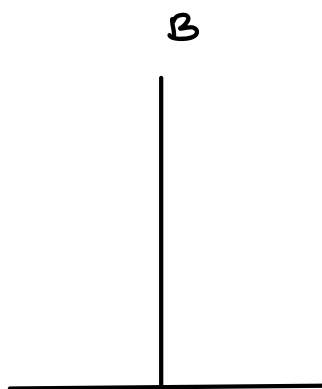
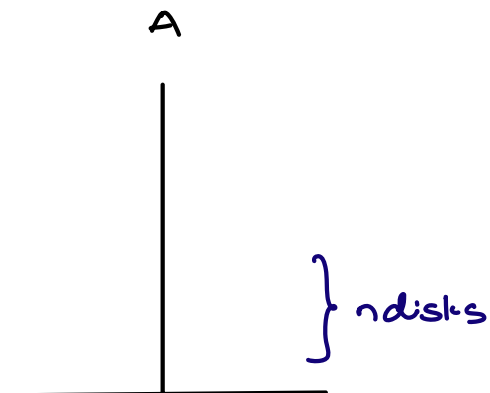
01. Move $n-1$ disk from $A \rightarrow B$



02 Move n^{th} disk from $A \rightarrow C$



03. Move $n-1$ disks from $B \rightarrow C$



void TOH (int n, A, B, C) ^{src} ^{dest}

$n \rightarrow$ no. of disks

if (n == 0) return;

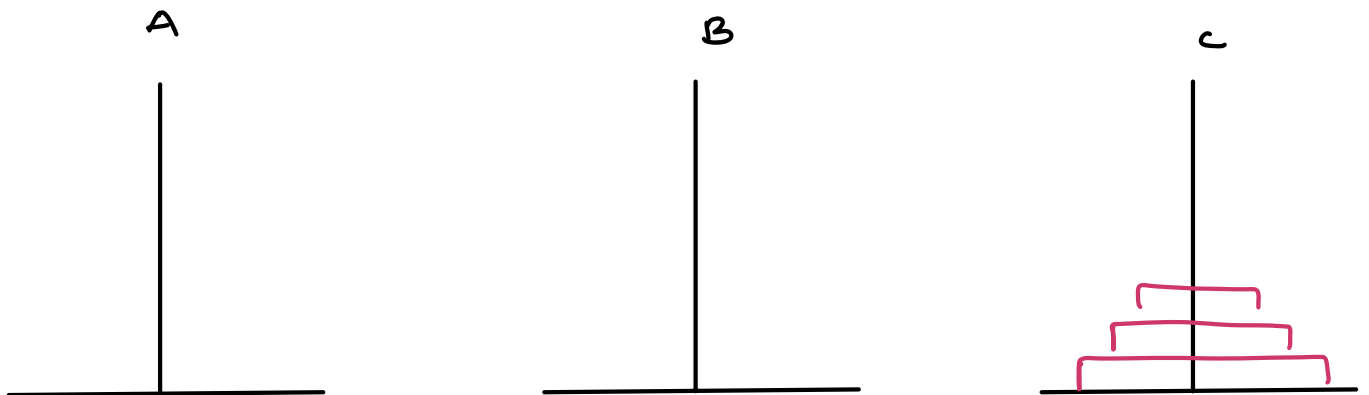
TOH (n-1, A, C, B) // move n-1 disk A $\rightarrow$ B

print(n: A $\rightarrow$ C) // move nth disk src $\rightarrow$ dest

TOH (n-1, B, A, C) // move n-1 disk B $\rightarrow$ C

}

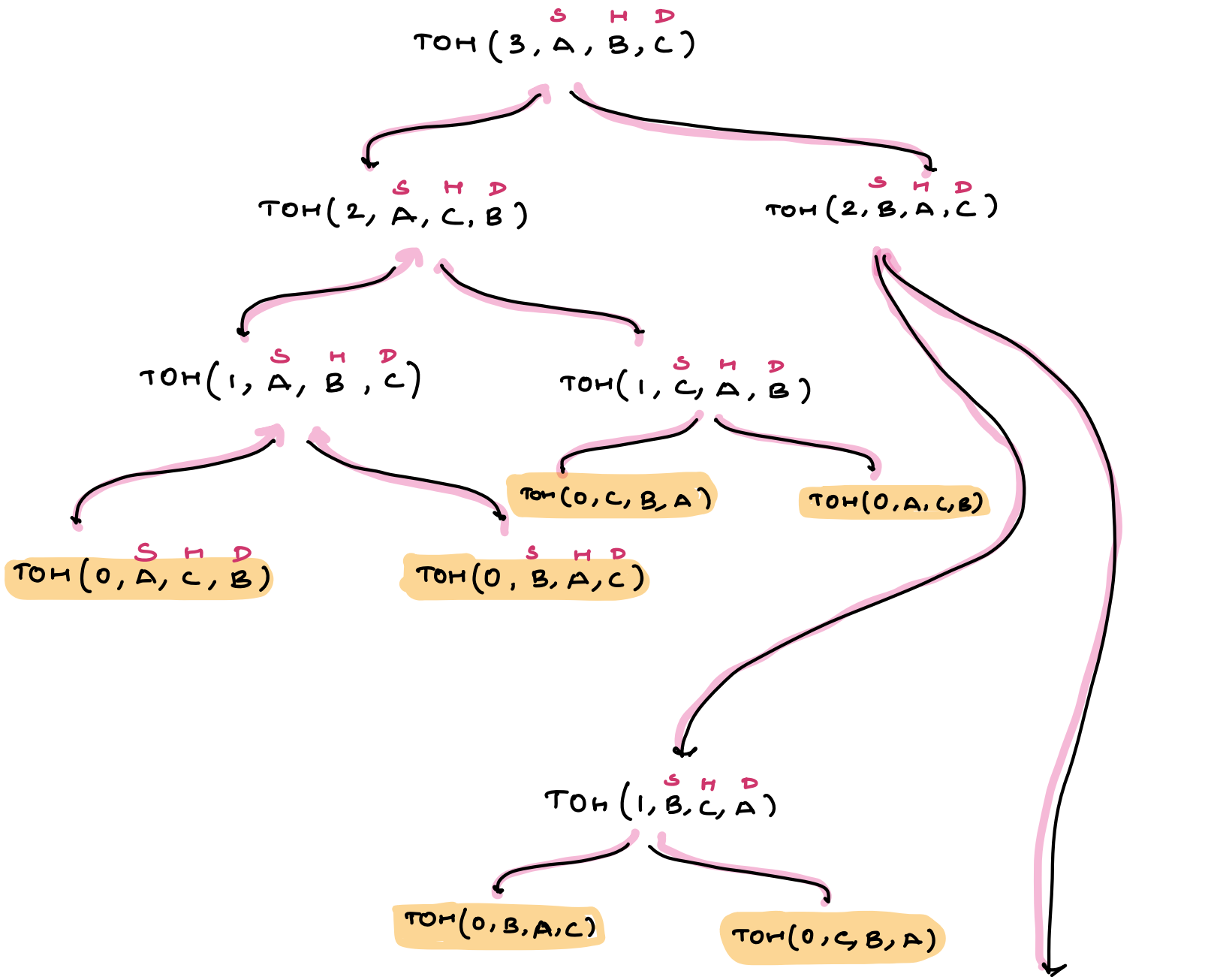
n = 3



Obs to create tree

Obs 1 = left call $\rightarrow$ (n-1, src, dest $\leftrightarrow$ helper)

Obs 2 = right call $\rightarrow$ (n-1, helper $\leftrightarrow$ src, dest)



Output :

- 1 : A → C
- 2 : A → B
- 1 : C → B
- 3 : A → C
- 1 : B → A
- 2 : B → C
- 1 : A → C

TC: $O(2^n)$
SC: $O(n)$

8:14 → 8:24 AM

Q Print all valid paranthesis of length $2N$
for a given value N

→ Equal no. of opening & closing brackets

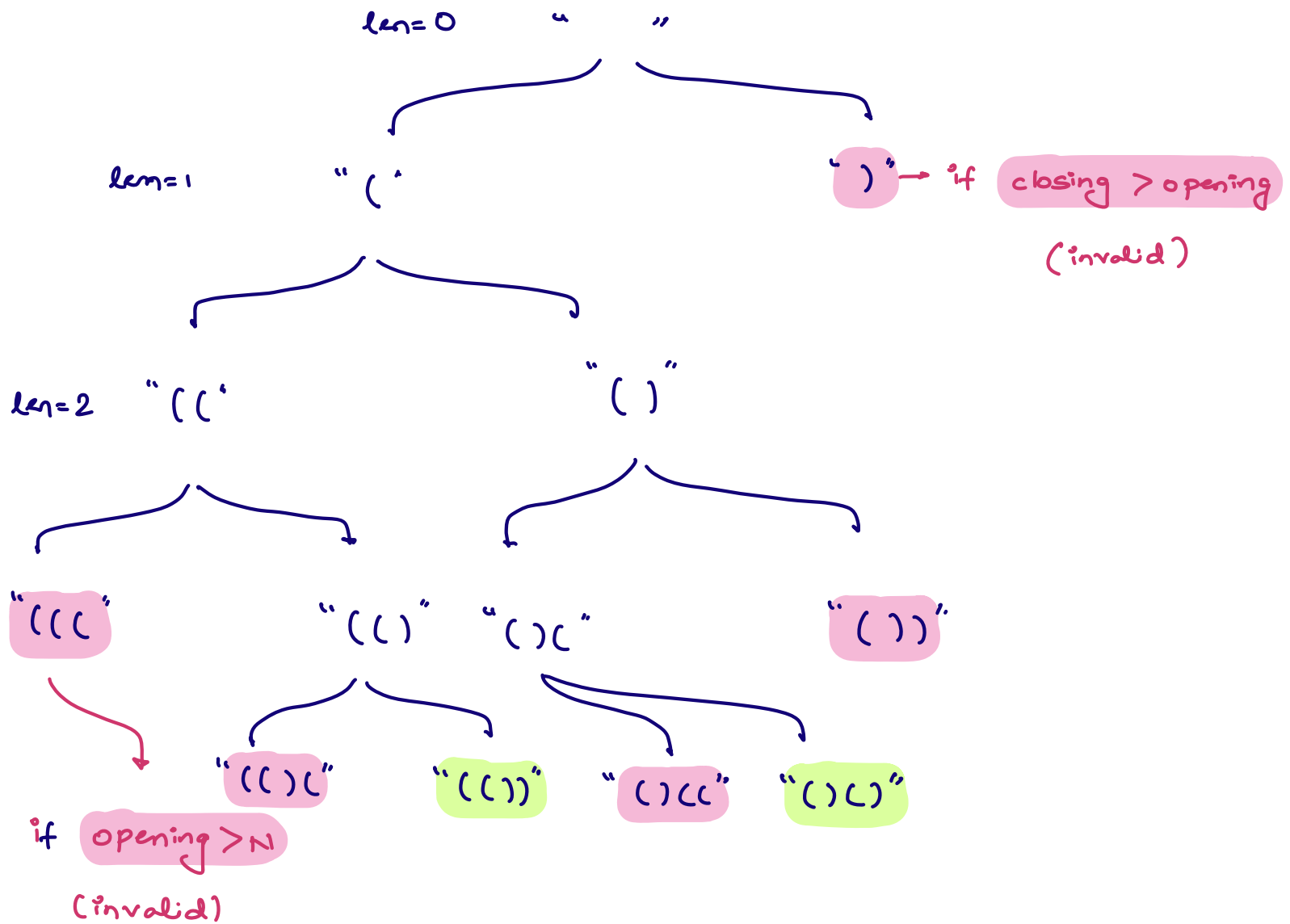
$N=1$ → ()) (

$N=2$ → () () (()))) (() () () () (())

$N=3$ → () () () ((())) (() () (()) () () () ()

* Idea → Start building your string from $n=0$ &
go till $2N$

$N=2$ → $2N=4$



```
void solve (N, str, open, closing)
```

```
if (str.length() == 2 * N) {
    print(str);
    return;
}
```

```
if (open < N)
```

```
    solve (N, str + "(", open + 1, closing);
```

```
if (close < open)
```

```
    solve (N, str + ")", open, close + 1);
```

TC: $O(2^n)$

SC: $O(2n) \approx O(n)$

$\text{solve}(2, " ", 0, 0)$

$\text{solve}(2, (, 1, 0)$

$\text{solve}(2, ((, 2, 0)$

$\text{solve}(2, ((, 2, 1)$

$\text{solve}(2, (((, 2, 2)$

$\text{solve}(2, (), 1, 1)$

$\text{solve}(2, ()(, 2, 1)$

$\text{solve}(2, ()(), 2, 2)$

Doubts

$N = 3$

```
void solve(N)
```

```
    if (N > 0) { ①
```

```
        solve(N-1) ②
```

```
        print(N) ③
```

```
    }
```

```
    println(); ④
```

```
}
```

Solve(3) 1 2

Solve(2) 1 2

Solve(1) 1 2 3 4

Solve(0) 1 4

Searching $\rightarrow O(n^2)$

Hash map

TC: $O(n)$

SC: $O(n)$

Sorting

TC: $O(n \log n)$

SC: —

Binary search

TC: $O(\log n)$

SC: $O(1)$

Two pointers

TC: $O(n)$

SC: $O(1)$

Subarray, continuous range sum $\rightarrow$ Pf sum, Cf, Contribution sliding